ĐHQG TP. Hồ Chí Minh
Trường Đại học Công nghệ Thông tin



Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT

Bảo vệ luận văn Thạc sĩ — Ngành Công nghệ Thông tin

Học viên: **Bùi Văn Dũng** — MSHV 220201014

TP. Hồ Chí Minh, 2026

Cán bộ hướng dẫn: **TS. Nguyễn Văn Dũ** · **TS. Đỗ Trí Nhựt**

Nội dung trình bày

I. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU NGHIÊN CỨU

II. PHƯƠNG PHÁP THỰC HIỆN

III. KẾT QUẢ 1 — ĐÁNH GIÁ OFFLINE LOẠI TRỪ RÒ RỈ

IV. KẾT QUẢ 2 — TRIỂN KHAI PHÂN TÁN TRÊN CỤM BIÊN

V. TRẢ LỜI CÂU HỎI NGHIÊN CỨU VÀ KẾT LUẬN

I. ĐẶT VẤN ĐỀ VÀ MỤC TIÊU NGHIÊN CỨU

Bối cảnh và ba khoảng trống nghiên cứu

Vấn đề thực tiễn

- Thiết bị IoT nhiều, tài nguyên ít, bảo mật yếu.
- Lưu lượng lớn → cần nền tảng **phân tán**.
- Cần phát hiện **ngay tại biên** để giảm độ trễ.

Ba khoảng trống

1. RF, SHAP, PCA chưa được so sánh trên *cùng một* pipeline.
2. Điểm số CICIDS2017 thường **gần như hoàn hảo** — dấu hiệu rò rỉ.
3. IDS biên hầu như chỉ đo trên **một thiết bị**.

Cách tiếp cận: hai pha bổ trợ

(A) Offline — chọn mô hình nào, bao nhiêu đặc trưng.

(B) Biên — mô hình đó có chạy thời gian thực được không?

Mục tiêu và ba câu hỏi nghiên cứu

Mục tiêu. Xây dựng hệ thống IDS trên Apache Spark, đánh giá dưới quy trình **loại trừ rò rỉ dữ liệu**, và kiểm chứng khả năng chạy thời gian thực trên cụm thiết bị biên.

RQ1 — Giảm chiều

Cắt hơn 60% đặc trưng có giữ được hiệu năng không?

RQ2 — Học tổ hợp

Có hơn mô hình đơn tốt nhất một cách *đáng kể* không?

RQ3 — Khả thi ở biên

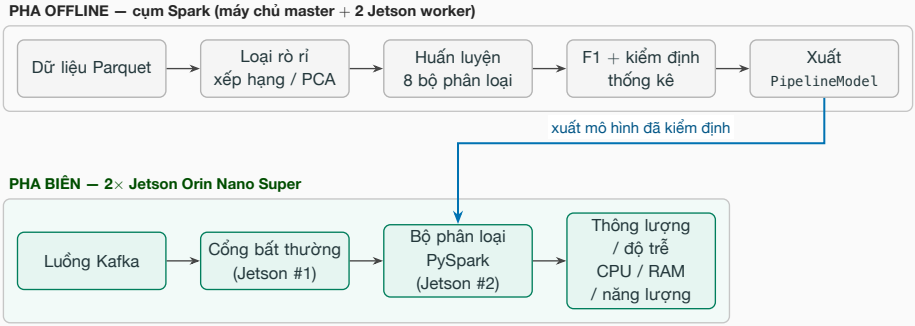
Chạy được thời gian thực trên cụm Jetson không?

Phạm vi. CICIDS2017[1], phân loại nhị phân, cụm 2 Jetson Orin Nano Super. Bộ dữ liệu *không* có lưu lượng IoT đặc thù — tính "IoT" nằm ở **kịch bản triển khai biên**.

[1] Sharafaldin, Lashkari & Ghorbani, *Toward generating a new intrusion detection dataset…*, ICISSP 2018.

II. PHƯƠNG PHÁP THỰC HIỆN

Kiến trúc tổng thể: hai pha của cùng một pipeline



Hai pha *không cạnh tranh* nhau: pha offline trả lời “mô hình nào, bao nhiêu đặc trưng”, còn pha biên trả lời “mô hình đó có chạy được theo thời gian thực dưới ràng buộc bộ nhớ, nhiệt và băng thông hay không”.

Pha A — Quy trình loại trừ rò rỉ và bốn chiến lược so sánh

Hai nguồn rò rỉ, xử lý *trước khi chia tập*

1. **Đặc trưng rò rỉ nhãn**: `destination_port` mã hóa thẳng dịch vụ bị tấn công.
2. **Trùng lặp mức đặc trưng**: cùng một mẫu lọt vào cả tập huấn luyện lẫn tập kiểm tra.

Nhóm *nhãn xung đột* bị **loại cả nhóm**: 270 nhóm, 44.260 dòng. Còn 77 đặc trưng.

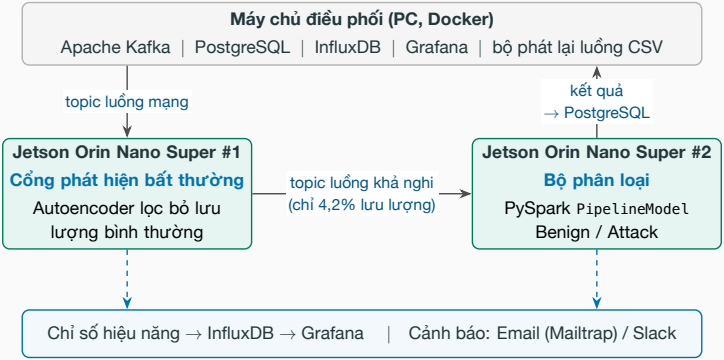
Bốn chiến lược, cùng một pipeline

- Baseline 77 đặc trưng · RF Top-K · SHAP Top-K · PCA k=40.
- 8 thuật toán + **Soft Voting**, **Hybrid Bagging**.

Chống thiên lệch: cấu hình cố định trước; tập kiểm tra **không tham gia** khâu lựa chọn nào.

Xếp hạng đặc trưng: RF importance — Breiman, *Machine Learning* 45(1), 2001; SHAP — Lundberg & Lee, NeurIPS 2017.

Pha B — Kiến trúc triển khai phân tán trên cụm biên



Ba chế độ được đo: A tách pipeline (đề xuất) · **B** mở rộng ngang · **C** cụm Spark. Mốc tham chiếu là đơn nút. Thông lượng đếm **số luồng xử lý xong mỗi giây** — mỗi luồng chỉ tính *một lần*, tại nơi có kết luận.

III. KẾT QUẢ 1 — ĐÁNH GIÁ OFFLINE LOẠI TRỪ RÒ RỈ

KQ1 — Giảm 60% đặc trưng mà vẫn giữ hiệu năng

Phương pháp	Mô hình	F1	Huấn luyện (s)	Đặc trưng
Baseline	XGBoost	0,9971	4615,5	77
SHAP Top-30	XGBoost	**0,9970**	**1226,1**	**30**
PCA k=40	XGBoost	0,9939	1444,8	40
RF Top-30	XGBoost	0,9922	1295,1	30

Thuật toán	RF Top-30	SHAP Top-30	Chênh lệch tương đối
Random Forest	0,9879	0,9955	+**0,77%**
XGBoost	0,9922	0,9970	+**0,48%**
GBT	0,9844	0,9918	+**0,75%**
Decision Tree	0,9835	0,9948	+**1,15%**

- SHAP Top-30 giữ F1 ngang baseline, cắt **60% đặc trưng** và **73% thời gian huấn luyện**.
- Cùng số đặc trưng, SHAP vượt RF Importance trên *mọi* thuật toán họ cây.
- Học tổ hợp *không* vượt được XGBoost đơn — xem slide sau.

KQ2 — Học tổ hợp không vượt được mô hình đơn tốt nhất

Trên SHAP Top-30 (cấu hình triển khai)	F1	Huấn luyện (s)	Dung lượng (MB)
XGBoost (mô hình đơn)	**0,9970**	**1226**	—
Soft Voting	0,9967	—	0,01
Hybrid Bagging	0,9966	**4909**	**7,15**

- Học tổ hợp chỉ thắng ở cấu hình mà mô hình đơn *yếu*: RF Top-30 (0,9938 so với 0,9922) và PCA (0,9940 so với 0,9939).
- Ở hai cấu hình mạnh nhất (Baseline, SHAP Top-30) thì **thua** mô hình đơn — nó *bù cho bộ đặc trưng kém*, không tạo thêm hiệu năng.

Trả lời **RQ2**: không có lợi thế đáng kể. Với thiết bị biên, mô hình đơn thắng cả về F1 lẫn chi phí — huấn luyện nhanh gấp 4 lần, mô hình nhỏ hơn nhiều.

KQ3 — Kiểm định thống kê thay vì so sánh một lần đo

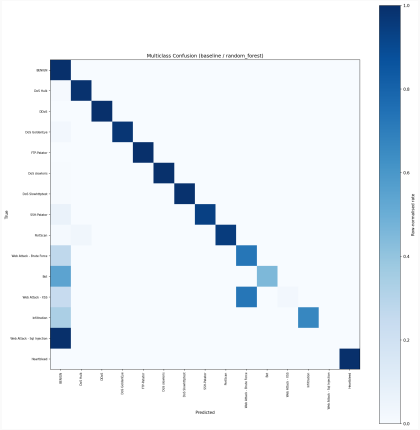
Phương pháp	Mô hình	F1 (TB $\pm$ CI95 t)
Baseline	XGBoost	0,99734 $\pm$ 0,00007
SHAP Top-30	XGBoost	0,99720 $\pm$ 0,00009
hoán vị ghép cặp $p = 0{,}031$; $d \approx 1{,}94$		
TOST: $p = 3{,}9 \times 10^{-6}$ (biên $\pm$0,001)		

Ảnh hưởng đặc trưng cổng (RF)	F1	Recall (Attack)
Giữ `destination_port`	0,9965	0,9988
Loại đặc trưng cổng	0,9955	0,9961

- N=6 lần *lấy mẫu lại* ghép cặp, không chỉ đổi seed.
- Hoán vị hai phía **liệt kê đầy đủ** 2^6 dấu: p nhỏ nhất là $2/2^6 \rightarrow$ **bắt buộc** $N \geq 6$.
- Lấy mẫu *chồng lấn* $\rightarrow$ kết luận theo khoảng tin cậy **Nadeau–Bengio**.
- **TOST**: chênh lệch thật $< \pm 2{,}3 \times 10^{-4}$ F1.

TOST bác bỏ được khác biệt $\geq$0,001 F1 $\rightarrow$ hai cấu hình **tương đương** theo nghĩa khẳng định. Tiêu chí phụ mới quyết định.

KQ4 — Đánh giá đa lớp: F1 nhị phân che giấu điều quan trọng



Lớp	Số mẫu	Recall	F1
Benign	380.119	0,9997	0,9989
DoS Hulk	34.446	0,9904	0,9947
Bot	290	**0,4552**	0,6256
Web – XSS	111	**0,0270**	0,0526
Web – SQL Injection	6	**0,0000**	0,0000
weighted-F1 = 0,998		**macro-F1 = 0,806**	

Hai kiểu suy giảm khác hẳn nhau:

- **Nhầm loại nhưng vẫn bắt được**: **76% XSS** vẫn bị gắn cờ ở mức nhị phân.
- **Lọt hẳn thành Benign** (nguy hiểm): **54% Bot**, cả 6 mẫu SQL Injection.

KQ5 — Tổng quát hóa liên bộ dữ liệu: điểm số không chuyển được

Huấn luyện → Kiểm tra	F1	AUC-PR
2017 → 2017	0,9953	0,9997
2017 → 2018	0,029–0,085 | 0,185–0,301	0,4849
2018 → 2018	0,9246	0,9679
2018 → 2017	0,0544	0,1926

Trung bình 18 lượt chạy; tập đặc trưng chung đã loại rò rỉ.

- Trong-miền ổn định *cả hai* chiều — liên bộ sụp: AUC-PR **1,00→0,48** và **0,97→0,19**.
- Recall 0,073 / 0,030: **không nhận ra** tấn công testbed kia (khác CICFlowMeter, cấu hình mạng, công cụ tấn công).
- 18 lượt, **hai chế độ rời nhau**: 11 lượt 0,029–0,085 và 7 lượt 0,185–0,301; AUC-PR lệch không đáng kể.

Điểm số trong-miền gần hoàn hảo — *kể cả khi đã loại rò rỉ* — **không chứng nhận** khả năng triển khai khi phân phối dịch chuyển.

IV. KẾT QUẢ 2 — TRIỂN KHAI PHÂN TÁN TRÊN CỤM BIÊN

KQ6 — Tách pipeline: thông lượng gấp 9 lần một nút

Chế độ	Số bo	Duy trì được (luồng/giây)	Độ trễ p50 / p95 (giây)	Bỏ qua ở cổng (%)	Năng lượng (J/dự đoán)
Một nút, không cổng	1	2,5	5,9 / 11,3	–	3,04
Một nút + cổng	1	≈2,5	3,8 / 7,8	89,0	3,04
B: Mở rộng ngang	2	3,3	4,1 / 8,2	–	3,20
C: Cụm Spark	2	9–17	0,7 / 5,2	88,6	1,15
A: Tách pipeline	2	**22,4**	1,0 / 7,1	88,8	**0,60**

- *Duy trì được* = tốc độ cao nhất mà **mọi luồng** đều có kết quả trong 10 giây; độ trễ tính từ lúc gửi, **gồm cả thời gian xếp hàng**.
- **A gấp 9 lần một nút** — nhờ đặt cổng và bộ phân loại trên *hai bo riêng*: thêm cổng trên cùng bo không nhanh hơn.
- **Cái giá**: cổng bỏ sót ≈30% luồng tấn công (recall thực đo 69,6%); p95 ≈7 giây vì mỗi lô PySpark mất vài giây.

KQ7 — Cái giá của việc đồng nhất một stack

Phục vụ suy luận bằng Spark để *đồng nhất với pha huấn luyện phân tán*. *Cùng một* rừng 200 cây, cùng 30 đặc trưng, cùng bo mạch, lô 10 luồng — ba engine cho dự đoán **trùng nhau 100%**:

Engine suy luận	Thông lượng (luồng/giây)	p95 một lô (ms)	Năng lượng (mJ/luồng)
PySpark (đang triển khai)	1,4	7515	1506
NumPy	54,8	194	11,3
ONNX Runtime	**10.092**	**1,2**	**0,087**

Mỗi lần gọi PySpark tốn ≈7 giây dù lô 10 hay 1.000 luồng; ở lô 10.000 khoảng cách còn ≈27 lần → **Q14**.

ONNX Runtime nhanh gấp **≈7.200 lần** PySpark trên cùng mô hình. Nút thắt của mọi chế độ ở KQ6 là engine PySpark, không phải mô hình → bước tiếp theo: xuất sang ONNX/TensorRT cho pha biên, giữ Spark cho pha huấn luyện.

Demo — Hệ thống chạy thật trên cụm hai Jetson

Video 1 · Pha A: huấn luyện phân tán (1:36)

1. Spark Master + Worker trên hai Jetson
2. Random Forest, SHAP Top-30, **toàn bộ** 1,79 triệu mẫu
3. Spark UI: job chạy trên cả hai bo

Kết quả: **F1 0,9957**, AUC-ROC 0,9999, huấn luyện 857 s.

► **Mở video pha A**

Video 2 · Pha B: triển khai biên (4:36)

1. Hạ tầng Kafka/PostgreSQL/Grafana, cổng lọc (#1), PySpark (#2)
2. Phát lại CICIDS2017 → log trực tiếp → Grafana
3. Đối chiếu từng dự đoán với nhãn thật

2.097 luồng: bỏ qua 88,8%, **recall 68,9%**, **0 báo động giả**.

► **Mở video pha B**

Lệnh và kết quả ghi từ **một lần chạy thật** (11/10/2026); số liệu khớp KQ1 và KQ6.

V. TRẢ LỜI CÂU HỎI NGHIÊN CỨU VÀ KẾT LUẬN

Trả lời trực tiếp ba câu hỏi nghiên cứu

RQ1 — Cắt hơn 60% đặc trưng có giữ được hiệu năng?

Có. SHAP Top-30 đạt F1 **0,9970** so với 0,9971 của baseline 77 đặc trưng, giảm ≈73% thời gian huấn luyện.

RQ2 — Học tổ hợp có hơn mô hình đơn tốt nhất một cách đáng kể?

Không. Trên SHAP Top-30, XGBoost đơn đạt **0,9970** còn Hybrid Bagging chỉ 0,9966 — thua, mà huấn luyện lâu gấp 4 lần.

RQ3 — Pipeline Spark có khả thi trên cụm thiết bị biên?

Khả thi cho cảnh báo gần thời gian thực, với điều kiện tách vai trò. Thông lượng **2,5** → **22,4** luồng/giây (gấp 9), năng lượng **3,04** → **0,60 J** mỗi dự đoán; đổi lại recall tấn công 69,6% và p95 ≈7 giây.

Đóng góp chính của luận văn

Về phương pháp luận

- **Quy trình đánh giá loại trừ rò rỉ**, khử trùng lặp *tất định* ở mức đặc trưng.
- **Kiểm định thống kê chặt chẽ**: hoán vị ghép cặp, khoảng tin cậy Nadeau–Bengio.
- **Đánh giá đa lớp và liên bộ dữ liệu** — chỉ ra điều F1 nhị phân che giấu.

Về hệ thống

- **Pipeline IDS hoàn chỉnh trên Apache Spark**, từ tiền xử lý đến suy luận thời gian thực.
- **Kiến trúc hai tầng trên cụm biên**: cổng bất thường + phân lớp PySpark, nối bằng Kafka.
- **Đo đạc vận hành đầy đủ**, kể cả năng lượng mỗi luồng.

Hạn chế và hướng phát triển

Hạn chế

- **Phân loại**: triển khai chính là nhị phân.
- **Dữ liệu**: CICIDS2017 không có lưu lượng IoT đặc thù.
- **Quy mô**: mới ở mức *minh chứng khái niệm*, cụm 2 nút.
- **Đối kháng**: chưa kiểm thử lưu lượng né tránh; GPU còn bỏ trống.

Hướng phát triển — ứng với từng hạn chế

- **Lớp tấn công hiếm**: cân bằng có chủ đích, kiến trúc phân tầng.
- **Dữ liệu IoT chuyên biệt**: Bot-IoT, TON_IoT, CIC-IoT-2023.
- **ONNX/TensorRT**, khai thác GPU Jetson.
- **Bền vững đối kháng**: hai bề mặt mới do cổng tạo ra.

Công trình khoa học từ luận văn

1. Thai Nguyen Vu, **Bui Van Dung**, Tri Nhut Do, Van Du Nguyen, *"A Comparison of Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML with a Leakage-Aware Evaluation Pipeline"*, Hội thảo quốc gia Nghiên cứu cơ bản và ứng dụng CNTT (**FAIR**), 2026. **(Accepted)**
2. Thai Nguyen Vu, **Dung Van Bui**, Nhut Tri Do, Van Du Nguyen, *"A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark"*, Symposium on Information and Communication Technology (**SOICT**), 2026. **(Accepted)**

Tài liệu tham khảo chính

[1] Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. *Toward generating a new intrusion detection dataset and intrusion traffic characterization*. ICISSP 2018. **CICIDS2017**

[2] Canadian Institute for Cybersecurity & CSE. *CSE-CIC-IDS2018 dataset*. 2018. **liên bộ dữ liệu**

[3] Lundberg, S. M. & Lee, S.-I. *A Unified Approach to Interpreting Model Predictions*. NeurIPS 2017. **SHAP**

[4] Breiman, L. *Random Forests*. Machine Learning 45(1), 2001. **RF**

[5] Chen, T. & Guestrin, C. *XGBoost: A Scalable Tree Boosting System*. KDD 2016. **XGBoost**

[6] Jolliffe, I. T. *Principal Component Analysis*, 2nd ed. Springer, 2002. **PCA**

[7] Zaharia, M. et al. *Apache Spark: A Unified Engine for Big Data Processing*. Communications of the ACM 59(11), 2016. **Spark**

[8] Kreps, J., Narkhede, N. & Rao, J. *Kafka: a Distributed Messaging System for Log Processing*. NetDB 2011. **Kafka**

Danh mục đầy đủ 71 mục nằm trong quyển luận văn.

Xin trân trọng cảm ơn Hội đồng

Em xin lắng nghe các ý kiến nhận xét và câu hỏi.

Q&A

Bấm vào tên slide để nhảy thẳng tới. Số hiệu Q*n* in ở tiêu đề từng slide.

Phạm vi và phương pháp

- **Q1** Phạm vi "IoT", mô hình mối đe dọa → **Phạm vi & đe dọa**
- **Q2** Khử trùng lặp, chống thiên lệch → **Chi tiết tiền xử lý**
- **Q3** Vì sao PCA k=40, vì sao loại LightGBM → **PCA & LightGBM**
- **Q4** Siêu tham số từng mô hình → **Siêu tham số**

Kết quả offline

- **Q5** So sánh đầy đủ 8 thuật toán × 4 phương pháp → **Bản đồ nhiệt F1**
- **Q6** Độ tin cậy, giới hạn kết luận → **Tính hợp lệ**

Kết luận

- **Q12** Kế hoạch tiếp theo, chi tiết → **Hướng phát triển**

Triển khai biên

- **Q7** Vì sao chọn Random Forest → **Chọn mô hình biên**
- **Q8** Cấu hình cụm, ba chế độ đo → **Chế độ & môi trường**, **Cụm huấn luyện**
- **Q9** Cách đo thông lượng, độ trễ, năng lượng → **Giao thức đo**
- **Q10** Ngưỡng cổng, đánh đổi recall → **Đường cong vận hành**
- **Q11** Bằng chứng chạy thực tế → **Grafana đỉnh tải**
- **Q13** Rò rỉ cổng theo từng mô hình → **Ablation 8 thuật toán**
- **Q14** Tăng kích thước lô có cứu được Spark? → **Quét kích thước lô**

Q1 — Phạm vi, dữ liệu và mô hình mối đe dọa

Phạm vi

- **Dữ liệu**: CICIDS2017, mở rộng sang CSE-CIC-IDS2018 cho thí nghiệm liên bộ dữ liệu.
- **Công nghệ**: Spark Core/SQL/Streaming/MLlib; Kafka, PostgreSQL, InfluxDB, Grafana.
- **Bài toán**: nhị phân Benign/Attack (có bổ sung đánh giá đa lớp theo loại tấn công).
- **Triển khai**: cụm 2 Jetson Orin Nano Super (8 GB) + máy chủ điều phối.

Lưu ý về phạm vi "IoT"

CICIDS2017 là lưu lượng doanh nghiệp *mô phỏng*, không chứa giao thức đặc thù IoT (MQTT, CoAP). Tính chất "IoT" nằm ở **kịch bản triển khai**: toàn bộ pipeline suy luận được đo trên *thiết bị biên tài nguyên hạn chế* — lớp gateway trong kiến trúc IoT ba tầng — chứ không ở bản chất lưu lượng huấn luyện.

Mô hình mối đe dọa

Kẻ tấn công ở bên trong hoặc ở biên mạng. Phạm vi giới hạn ở các tấn công đã biết kiểu CICIDS2017. Tấn công đối kháng (adversarial/evasion) **nằm ngoài phạm vi** đánh giá.

Q2 — Chi tiết tiền xử lý và chống thiên lệch lựa chọn

Kết quả tiền xử lý (tất định)

Nhóm nhãn mâu thuẫn bị loại	270 nhóm
Số dòng bị loại kèm theo	44.260
Đặc trưng số còn lại	77
Huấn luyện / kiểm tra	1,79 tr / 0,45 tr

Chống thiên lệch lựa chọn

- Tách *pha thăm dò* (quét *K*) khỏi *pha xác nhận* — 4 cấu hình cố định trước khi xem kết quả.
- Mô hình đại diện chọn theo F1 trên **tập kiểm định** tách từ tập huấn luyện.
- Tập kiểm tra **không tham gia** bất kỳ bước lựa chọn nào.
- Siêu tham số cố định, dùng chung cho mọi phương pháp.

Nhóm trùng đặc trưng nhưng *nhãn xung đột* (lỗi đã ghi nhận của CICIDS2017) bị loại **cả nhóm**, thay vì giữ một dòng tùy cách Spark phân vùng — nhờ vậy kết quả tất định, tái lập được.

Q3 — Vì sao PCA dùng k=40, và vì sao loại LightGBM?

PCA k=40 thay vì k=30

PCA[2] là phép *trích xuất*, không phải *chọn lọc*: mỗi thành phần chính là một tổ hợp tuyến tính chỉ mang một phần phương sai, nên cần nhiều thành phần hơn để giữ lượng thông tin tương đương tập 30 đặc trưng gốc. Trong dải quét chung $\{20, 30, 40\}$, k=40 là mức giữ phương sai cao nhất, nên là điểm vận hành đại diện *công bằng nhất* cho PCA. Ép k=30 sẽ làm PCA mất thêm phương sai và bất lợi khi so sánh.

LightGBM bị loại khỏi so sánh

Thư viện native của LightGBM (qua SynapseML) chỉ hỗ trợ kiến trúc x86_64, không chạy được trên thiết bị biên Jetson Orin Nano Super (ARM64) — vốn là *đích triển khai* của luận văn. Trong họ gradient boosting, XGBoost và GBT đóng vai trò đại diện.

---

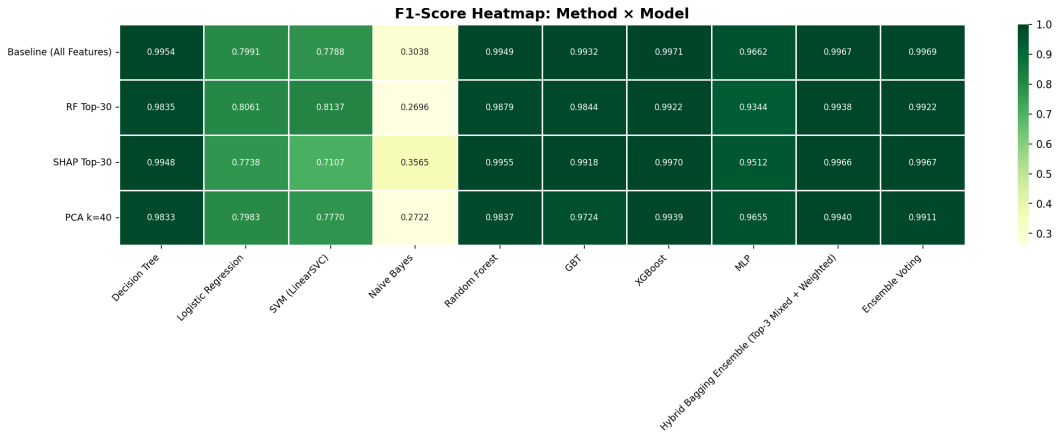
[2] Jolliffe, *Principal Component Analysis*, 2nd ed., Springer 2002.

Q4 — Siêu tham số cố định trước (a priori)

Mô hình	Siêu tham số chính
Decision Tree	maxDepth=15, minInstancesPerNode=5, impurity=entropy
Logistic Regression	maxIter=200, regParam=0.001, L2, threshold=0.5
SVM (LinearSVC)	maxIter=200, regParam=0.001
Naive Bayes	gaussian, smoothing=1.0
Random Forest	numTrees=200, maxDepth=15, featureSubset=sqrt
GBT	maxIter=150, maxDepth=6, stepSize=0.05, subsample=0.8
XGBoost	n_estimators=300, max_depth=8, lr=0.05, subsample=0.8
MLP	layers=[d,64,32,2], maxIter=150, stepSize=0.01

Dùng chung cho *mọi* phương pháp giảm chiều, để khác biệt quan sát được phản ánh đúng *phương pháp chọn đặc trưng*. Tối ưu siêu tham số (grid search + cross-validation) tách thành giai đoạn riêng, chỉ áp dụng cho cấu hình đã chọn — mức cải thiện nhỏ, phù hợp với nhận định bài toán nhị phân trong-miền đã gần bão hòa.

Q5 — Bản đồ nhiệt F1: phương pháp × thuật toán



Nhị phân, đã loại đặc trưng cổng. 8 thuật toán × 4 phương pháp.

Q6 — Các yếu tố ảnh hưởng tính hợp lệ

- **Nội tại.** Xếp hạng RF/SHAP lấy từ tập huấn luyện gốc và giữ cố định qua các lần lấy mẫu lại — nhất quán với thiết kế "cấu hình cố định trước", nhưng có thể gây rò rỉ nhẹ ở phần xếp hạng. Xếp hạng lại trong hai trong sáu lần lấy mẫu chỉ làm F1 lệch $\leq 7{,}4 \times 10^{-5}$, và 25–26/30 đặc trưng vẫn được chọn lại. MLP không được gán trọng số lớp (Spark ML không hỗ trợ).
- **Ngoại tại.** CICIDS2017 (2017) có thể không phản ánh lưu lượng hiện tại. Các bộ dữ liệu NIDS chuẩn đều có hạn chế chất lượng đã ghi nhận. Kết luận liên bộ dữ liệu dựa trên *mẫu phân tầng* của CSE-CIC-IDS2018. Chưa đánh giá trước lưu lượng đối kháng. Bản thân việc khử trùng lặp cũng làm thay đổi phân bố benign/attack.
- **Thống kê.** $N{=}6$ cho *power* thấp: kiểm định hai phía liệt kê đầy đủ chặn p ở mức $2/2^N$, nên $N{=}6$ chỉ vừa đủ chạm ngưỡng 0,05. Các lần lấy mẫu chồng lấn, vi phạm giả định độc lập của phân phối t — do đó Nadeau–Bengio là căn cứ chính. Tăng N với cross-validation lồng nhau là hướng củng cố tiếp theo.

Q7 — Vì sao chọn Random Forest cho thiết bị biên?

Ứng viên (giai đoạn chọn)	Thông lượng (mẫu/giây)	p95 (ms)	CPU (%)	RAM (%)
Random Forest	**26,8**	**397**	**35,2**	27,5
GBT	24,3	439	40,9	24,0
Decision Tree	22,0	537	46,4	**21,9**

Random Forest[3] thắng ở **cả ba chỉ số vận hành** quan trọng nhất với IDS thời gian thực, và có F1 cao nhất trong ba ứng viên (0,9955 ngoại tuyến). Với IDS, *độ trễ dự đoán là ràng buộc trực tiếp* — mô hình nhỏ nhưng chậm hơn 35% về p95 sẽ kéo dài cửa sổ mà tấn công chưa bị chặn.

Đánh đổi là bộ nhớ: 27,5% RAM và 4,4 MB so với 21,9% và 0,05 MB của Decision Tree — trên bo mạch 8 GB vẫn còn dư địa lớn. Decision Tree là *phương án dự phòng* cho thiết bị IoT eo hẹp hơn.

[3] Breiman, *Random Forests*, Machine Learning 45(1), 2001.

Q8 — Ba chế độ triển khai và môi trường đo

Đơn nút toàn bộ pipeline trong một tiến trình — mốc tham chiếu.

A: Tách pipeline cổng trên #1, phân loại trên #2 (**thiết kế đề xuất**).

B: Mở rộng ngang cả hai nút chạy *vai trò đầy đủ*, chia tải qua Kafka consumer group.

C: Cụm Spark máy chủ làm master, hai Jetson làm worker.

Thông lượng đếm **số luồng xử lý xong mỗi giây**: mỗi luồng tính *một lần*, ở cổng nếu bị bỏ qua, ở bộ phân loại nếu được chuyển tiếp — luồng qua cả hai nút không bị đếm hai lần.

Môi trường thực nghiệm

2× Jetson Orin Nano Super: CPU 6 nhân Arm Cortex-A78AE, GPU Ampere ≈67 TOPS, 8 GB LPDDR5, SSD NVMe 256 GB.

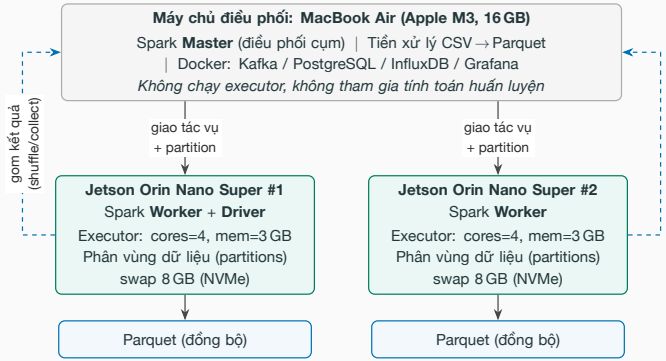
PySpark 3.4.1, Kafka 3.5, LAN Wi-Fi.

Tải: phát lại CICIDS2017 ở 100 luồng/giây.

≥3 lần lặp, lưu lượng khởi động *loại khỏi* cửa sổ đo.

Năng lượng: `tegrastats` — KQ6 toàn bo mạch, KQ7 phần hoạt động.

Q8 (tiếp) — Kiến trúc cụm huấn luyện phân tán Spark Standalone

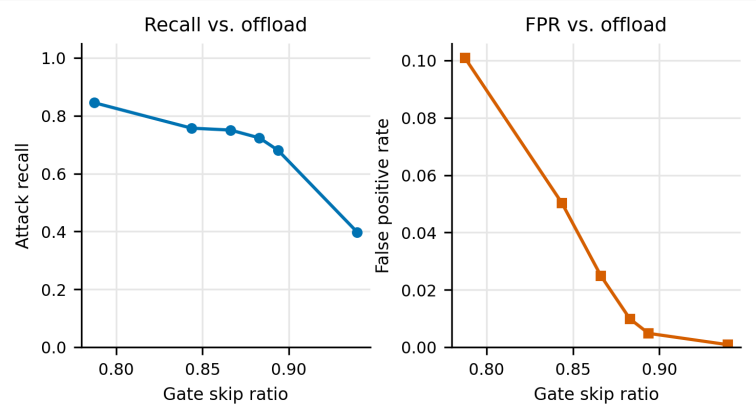


MacBook *chỉ* điều phối (master), không chạy executor, để thời gian huấn luyện phản ánh đúng **cụm biên thuần Jetson**; đưa M3 vào tính toán sẽ gây lệch tải (straggler).

Q9 — Giao thức đo hiệu năng ở pha biên

- Script điều phối phía máy chủ khởi động các vai trò pipeline trên Jetson qua SSH, sinh tải, thu thập log từng nút.
- $\geq$3 lần lặp mỗi cấu hình. Lưu lượng khởi động *loại khỏi* cửa sổ đo, để JVM/JIT warmup không làm phồng độ trễ batch đầu.
- Mọi truy vấn chỉ số canh đúng cửa sổ tải, không dùng khoảng thời gian trượt phía sau.
- Phân vị độ trễ tính *theo từng nút* trên mẫu thô từng luồng, còn p95 mức cụm lấy *thận trọng* theo nút tệ nhất.
- Độ trễ đầu cuối dựa trên đồng hồ đồng bộ NTP và *không* bao gồm chi phí ghi cơ sở dữ liệu.
- Năng lượng: `tegrastats`, 30 giây không tải rồi đến cửa sổ tải; chế độ hai nút cộng công suất cả hai bo mạch.

Q10 — Cổng bất thường là một *đường cong vận hành*



Recall tấn công (trái) và tỷ lệ báo động giả (phải) theo tỷ lệ bỏ qua tại cổng. Nâng tỷ lệ bỏ qua giúp giảm tải Spark nhưng đánh đổi bằng recall.

Q13 — Rò rỉ cổng đích phụ thuộc họ mô hình

Thuật toán	F1 (loại cổng)	F1 (giữ cổng)	ΔF1
SVM (LinearSVC)	0,7859	0,8274	**+0,0415**
Logistic Regression	0,7986	0,8381	**+0,0395**
Naive Bayes	0,2985	0,3093	+0,0108
MLP	0,9660	0,9711	+0,0052
GBT	0,9924	0,9941	**+0,0017**
Decision Tree	0,9953	0,9966	**+0,0014**
Random Forest	0,9954	0,9964	**+0,0010**
XGBoost	0,9972	0,9979	**+0,0007**

Hai lượt trên cùng một phép chia. Mức thổi phồng trải gần hai bậc độ lớn → **không suy được mức rò rỉ của lớp mô hình này từ mô hình khác**.

Q11 — Bất đối xứng tải lúc đỉnh tải (Grafana)

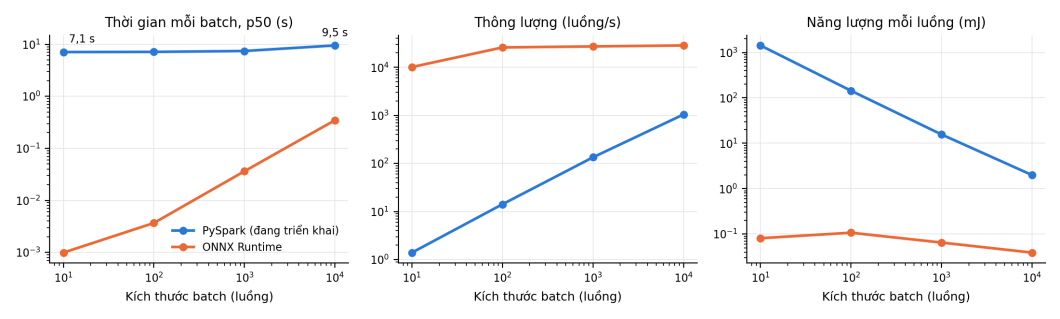


Tại đỉnh tải, chế độ tách pipeline (lượt chạy trước, mô hình cũ):

- nút cổng: **4,6% CPU**
- nút phân loại: **90,8% CPU**

Bộ phân loại chiếm gần như toàn bộ ngân sách tính toán — chính vì vậy việc đưa nó ra khỏi đường ingest mới mang lại hiệu quả.

Q14 — Spark tốn ở mỗi lần gọi, không phải ở các cây



Cùng rừng 200 cây, cùng bo Jetson #2, lô từ 10 tới 10.000 luồng, dự đoán trùng nhau 100%. Một lần gọi PySpark mất 7,1–7,4 giây tới lô 1.000 và 9,5 giây ở lô 10.000; ONNX bão hòa quanh 28.000 luồng/giây. Khoảng cách giảm từ ≈7.200 lần (lô 10) xuống ≈27 lần (lô 10.000).

Q12 — Hướng phát triển chi tiết

- **Cải thiện độ nhạy cho lớp tấn công hiếm.** Xử lý mất cân bằng có chủ đích (undersampling/SMOTE áp dụng *sau* khi chia tập), focal loss, hạ ngưỡng quyết định theo lớp; **kiến trúc phân tầng** — tận dụng chính thiết kế cổng hai tầng: tầng nhị phân ở biên giữ recall tổng, còn bộ định danh loại chỉ huấn luyện trên *luồng tấn công*, huấn luyện lại ngoại tuyến mà không đụng hệ thống đang chạy; thêm **đặc trưng chuỗi thời gian theo host** cho beaconing C&C.
- **Kiểm chứng trên dữ liệu IoT chuyên biệt.** Bot-IoT, TON_IoT, CIC-IoT-2023; mở rộng thí nghiệm liên bộ dữ liệu lên toàn bộ CSE-CIC-IDS2018.
- **Tối ưu suy luận biên bằng ONNX/TensorRT.** Tích hợp phiên suy luận ONNX vào pipeline streaming, tăng tốc bằng TensorRT trên GPU Jetson; đo độ trễ đầu cuối đầy đủ.
- **Bền vững trước tấn công đối kháng.** Đặc biệt hai bề mặt tấn công mới do cổng tạo ra: giả dạng lưu lượng bình thường để vượt cổng, và cố ý tạo sai số tái tạo cao để dồn tải lên nút phân loại.